

Project Design Phase-II

Technology Stack (Architecture & Stack)

Date	13 July 2026
Team ID	
Project Name	Voice-Based Concept Understanding Analyzer (VBCUA)
Maximum Marks	4 Marks

Technical Architecture:

The deliverable below includes the architectural diagram of the implemented system, followed by the component and characteristic details in Table 1 and Table 2.

Application: Voice-Based Concept Understanding Analyzer (VBCUA) — local, single-process deployment.

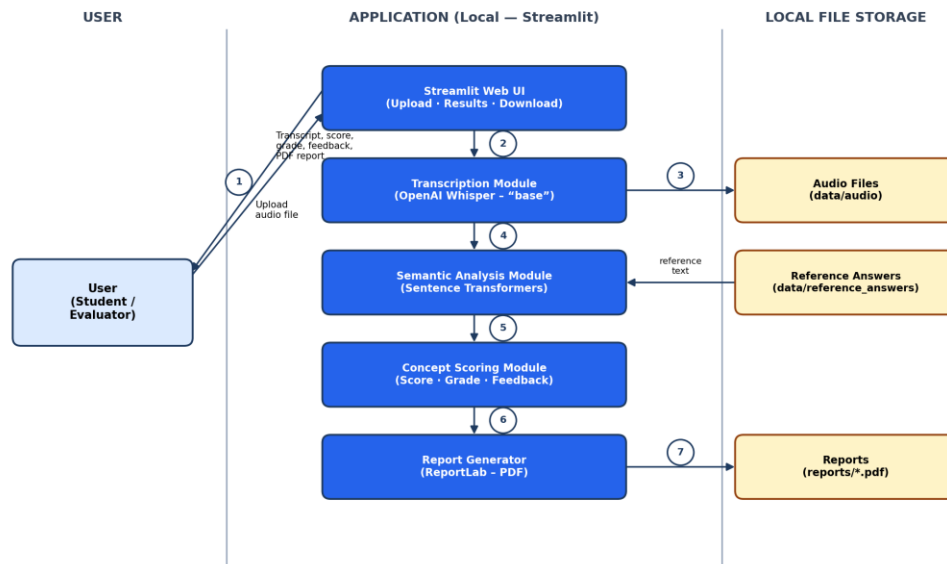


Figure 1: VBCUA Technical Architecture (Local, Single-Process Deployment)

Table-1: Components & Technologies

S.No	Component	Description	Technology
1.	User Interface	How the user interacts with the application: uploading an audio file and viewing the transcript, score, grade, feedback, and report.	Streamlit (Python-based web UI framework)
2.	Application Logic-1	Converts the uploaded spoken response into text.	OpenAI Whisper ("base" model)
3.	Application Logic-2	Compares the transcript with the reference answer to compute a semantic similarity score.	Sentence Transformers (all-MiniLM-L6-v2), PyTorch
4.	Application Logic-3	Converts the similarity score into a percentage, assigns a grade tier, and generates feedback text.	Python (rule-based scoring logic in concept_scorer.py)

S.No	Component	Description	Technology
5.	Database	Not implemented in the current version. The application does not use a relational or NoSQL database; only in-memory Streamlit session state is used during a session.	Not Applicable
6.	Cloud Database	Not implemented in the current version. No cloud-hosted database service is used.	Not Applicable
7.	File Storage	Storage of uploaded audio files, the reference answer text file, and generated PDF reports.	Local filesystem (data/audio, data/reference_answers, reports/)
8.	External API-1	Not implemented in the current version. No third-party or cloud AI API is called; Whisper and Sentence Transformers run locally.	Not Applicable
9.	External API-2	Not implemented in the current version.	Not Applicable
10.	Machine Learning Model	Automatic speech recognition (audio-to-text) and semantic embedding generation for similarity comparison.	OpenAI Whisper ("base") and Sentence Transformers (all-MiniLM-L6-v2)
11.	Infrastructure (Server / Cloud)	Local Server Configuration: runs as a single local Streamlit process on the developer/user machine (python -m streamlit run app/main.py). Cloud Server Configuration: not implemented in the current version; cloud deployment is identified as future scope.	Local (Streamlit development server)

Table-2: Application Characteristics

S.No	Characteristics	Description	Technology
1.	Open-Source Frameworks	The application is built entirely on open-source Python libraries for the web interface, speech recognition, semantic analysis, and report generation.	Streamlit, OpenAI Whisper, Sentence Transformers, PyTorch, ReportLab
2.	Security Implementations	Uploaded files are restricted to a defined set of audio formats before processing, and inputs are validated at each pipeline stage (e.g., empty transcript, missing reference file). Authentication, encryption, and access control are not implemented in the current version.	Streamlit file-type validation; Python input/exception validation
3.	Scalable Architecture	The application follows a single-process, modular monolith design (separate modules per pipeline stage) rather than a micro-services or multi-tier architecture. It is designed for single-user, single-session use; concurrent multi-user scaling is identified as future scope.	Modular Python package structure (app/modules, app/ui, app/utils)
4.	Availability	The application runs as a single local Streamlit server process. Availability depends on the host machine remaining online; no load balancing or distributed deployment is currently implemented.	Single local Streamlit process
5.	Performance	The Whisper and Sentence Transformer models are loaded once and cached in memory (module-level singleton pattern) to avoid repeated model-loading overhead across analyses within the same session.	In-memory model caching (global variables in transcriber.py and semantic_analyzer.py)

References:

<https://c4model.com/>

<https://www.ibm.com/cloud/architecture>

<https://aws.amazon.com/architecture>

<https://medium.com/the-internal-startup/how-to-draw-useful-technical-architecture-diagrams-2d20c9fda90d>